



Aspectos prácticos del Entrenamiento (I)

Redes Neuronales y Aprendizaje Profundo

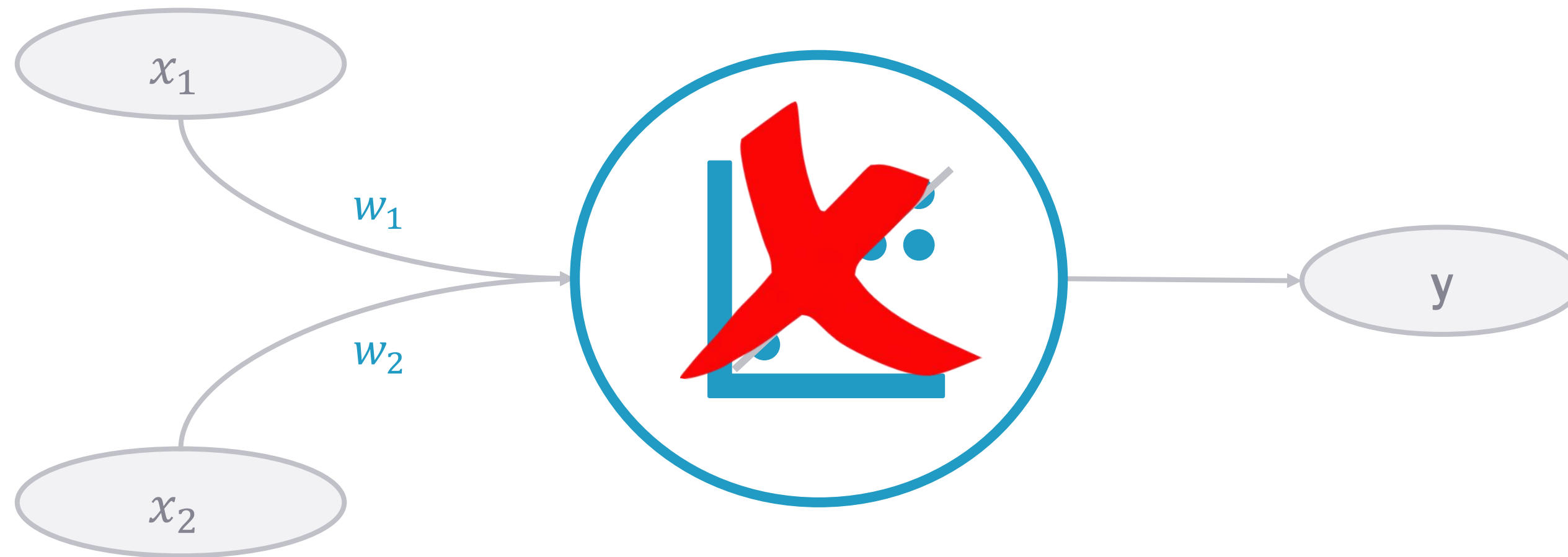
00

En anteriores
episodios...

Disposición de neuronas

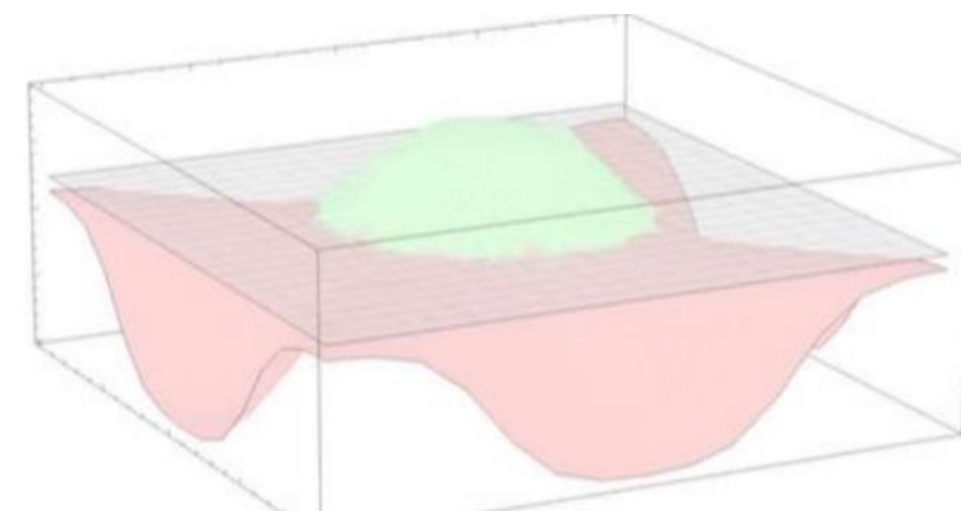
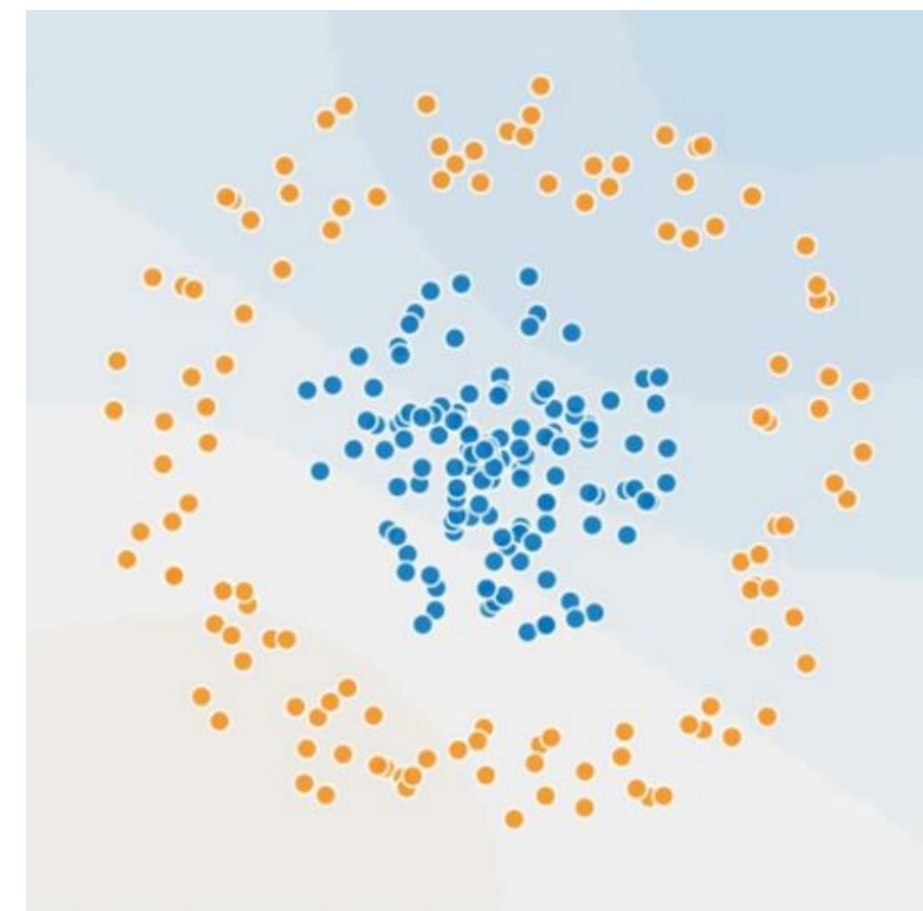
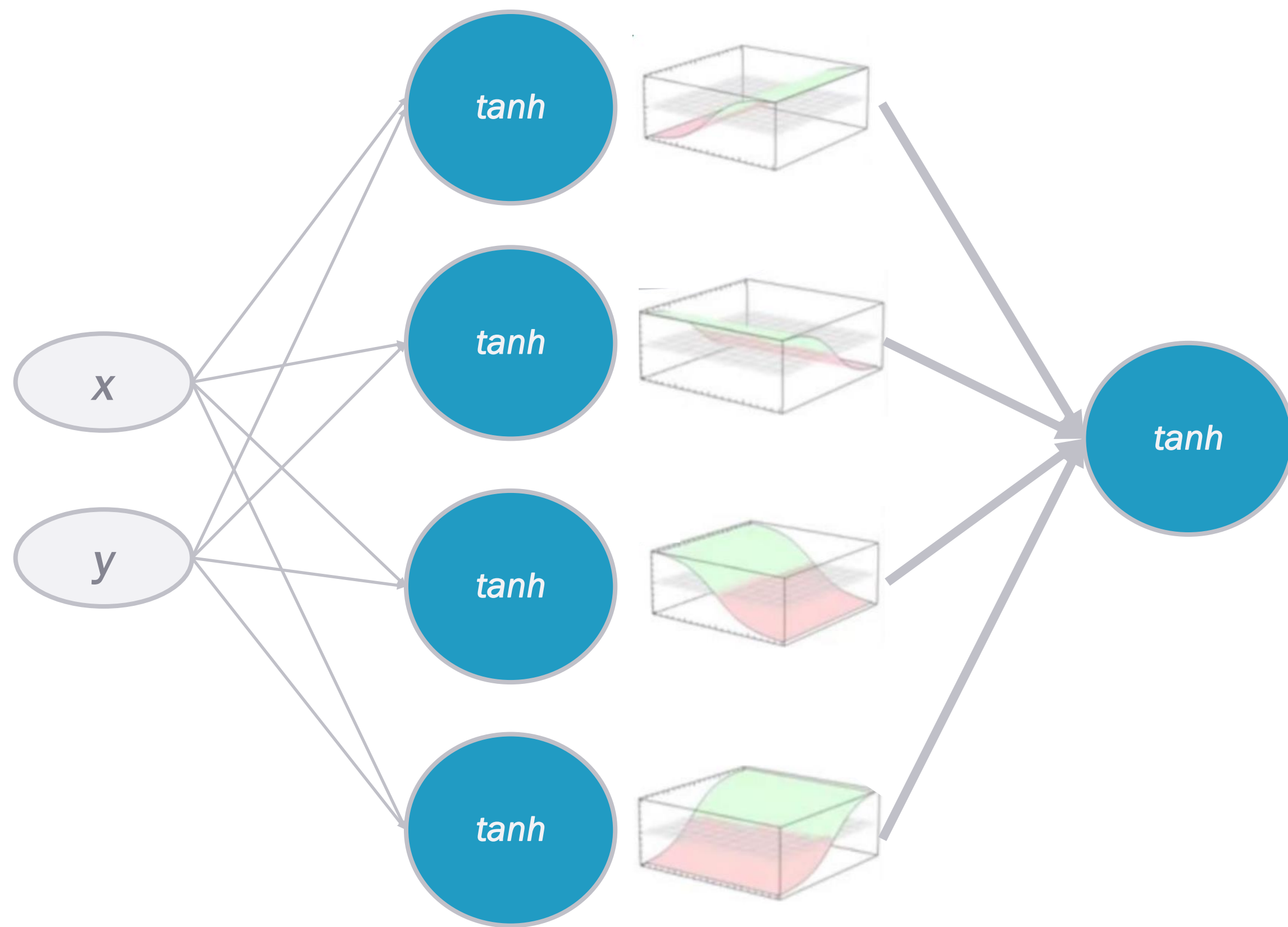


Funciones de activación



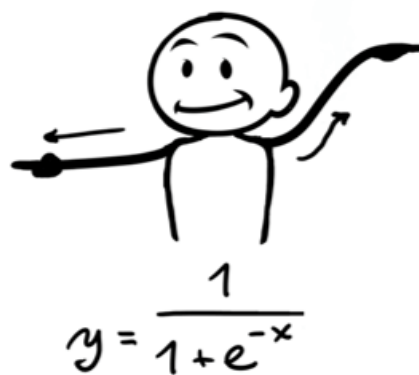
$$x_1 w_1 + x_2 w_2 + b$$

$$f(x_1 w_1 + x_2 w_2 + b)$$



Otras funciones

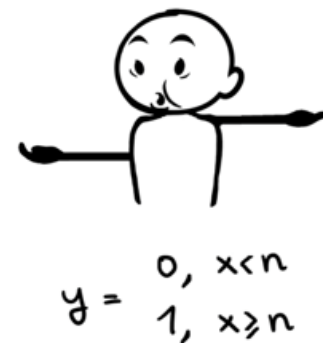
Sigmoid



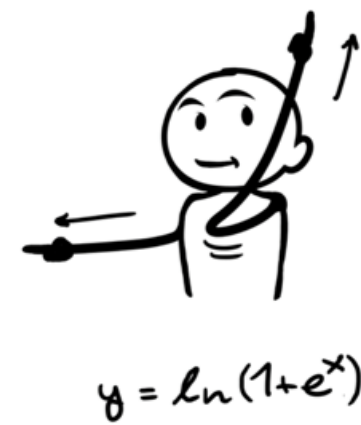
Tanh



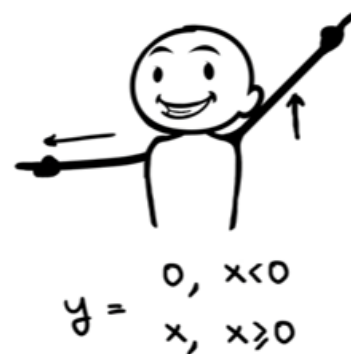
Step Function



Softplus



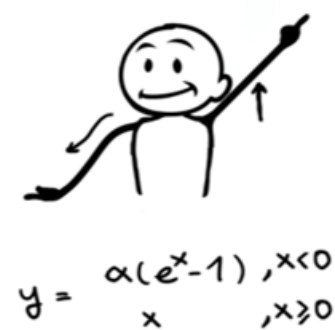
ReLU



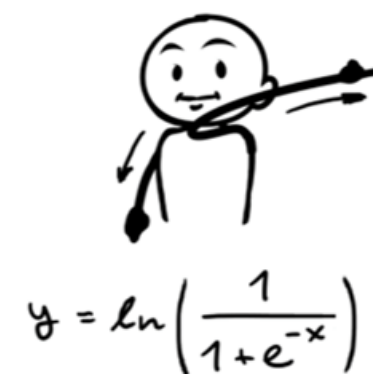
Softsign



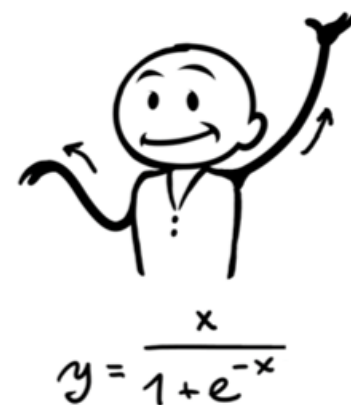
ELU



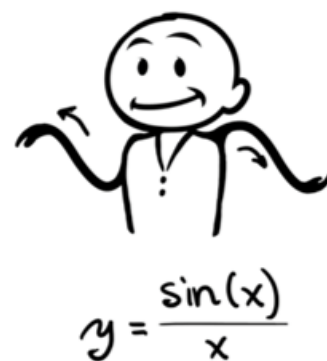
Log of Sigmoid



Swish



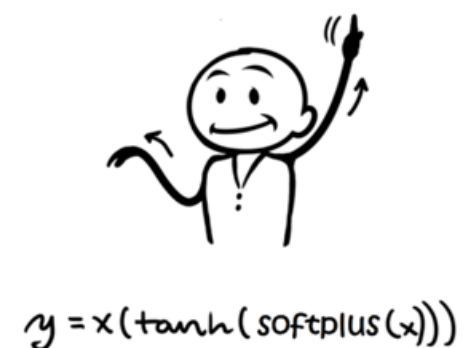
Sinc



Leaky ReLU



Mish

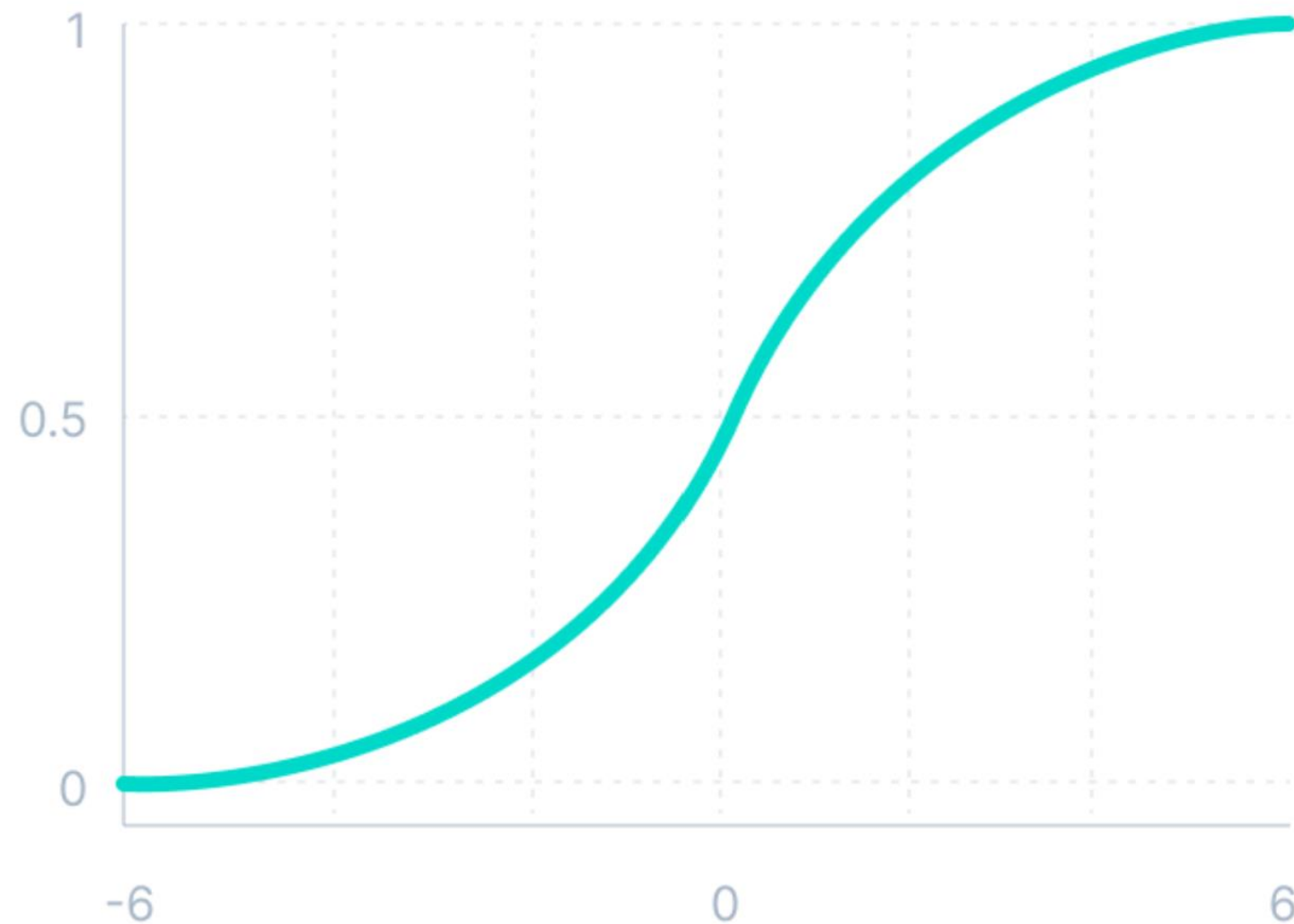


01

Funciones de Activación

Función Sigmoide

$$f(x) = \frac{1}{1 + e^{-x}}$$



PROs

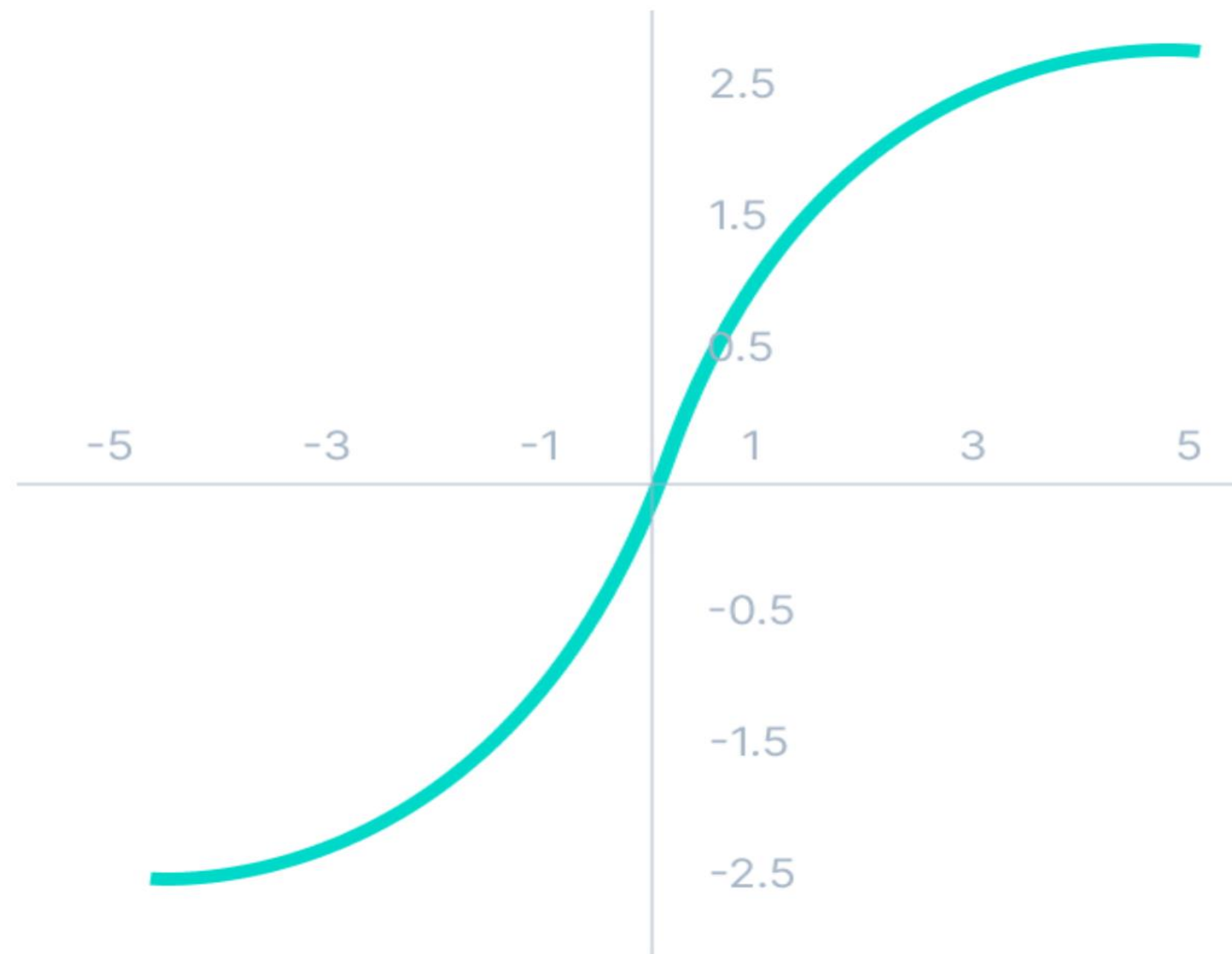
- Salida normalizada entre 0 y 1

CONs

- Desvanecimiento de gradiente: a valores grandes, poca diferencia
- Siempre es positivo; no está centrado en 0
- Operación computacionalmente costosa

Función Tangente Hiperbólica

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$



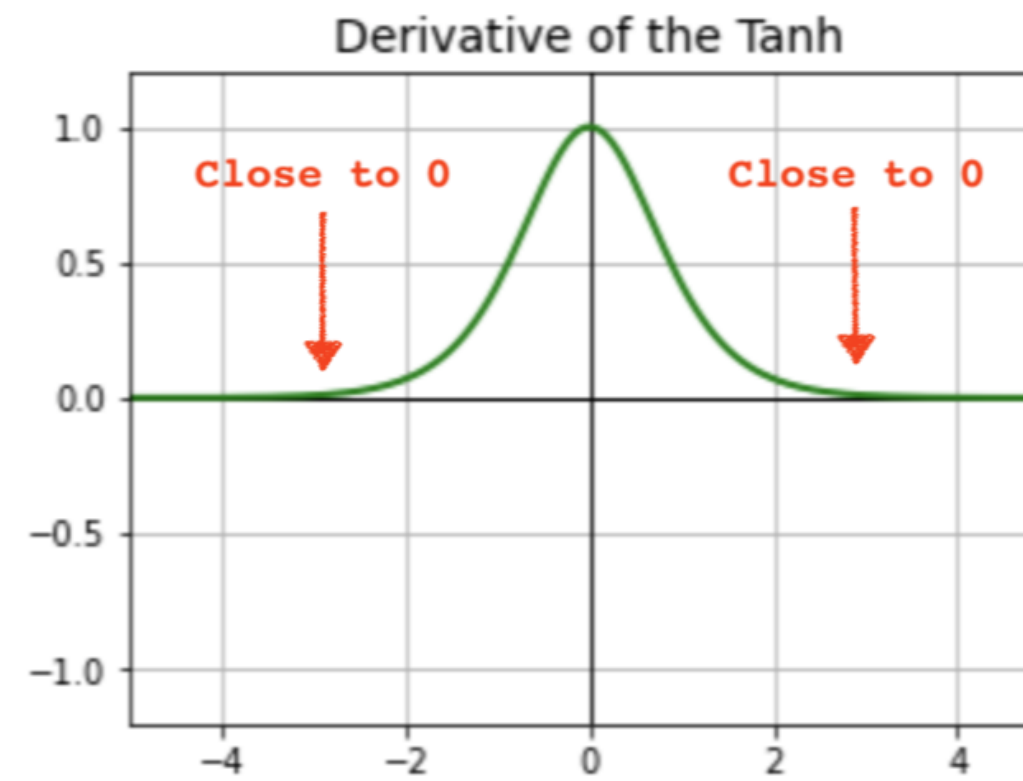
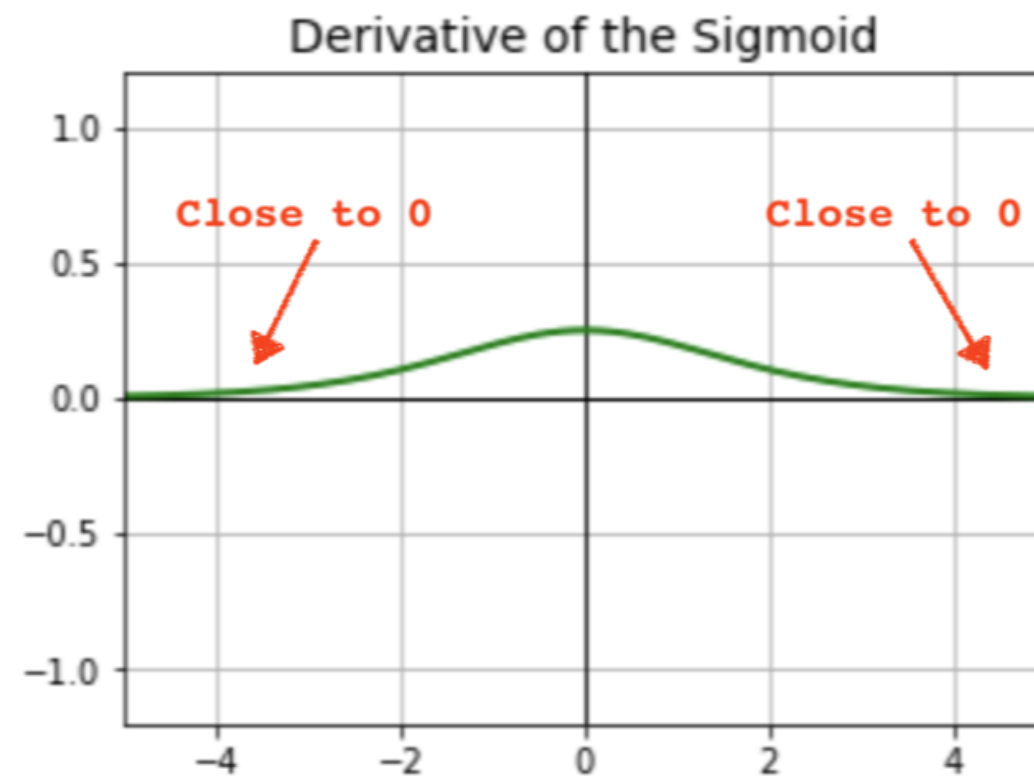
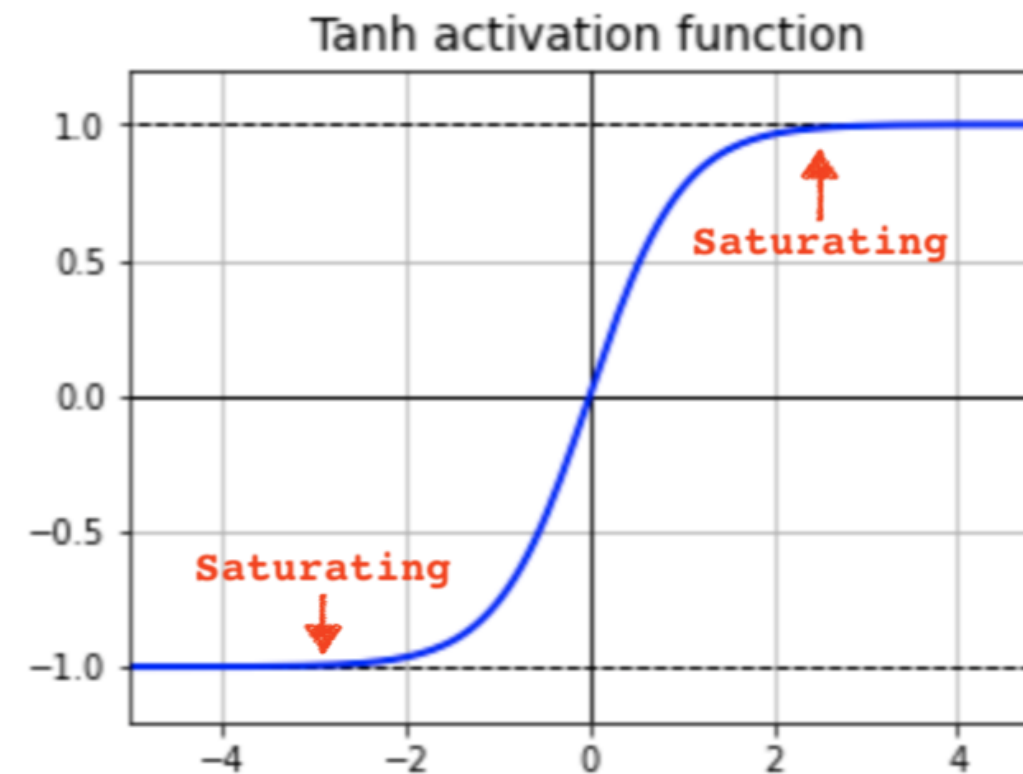
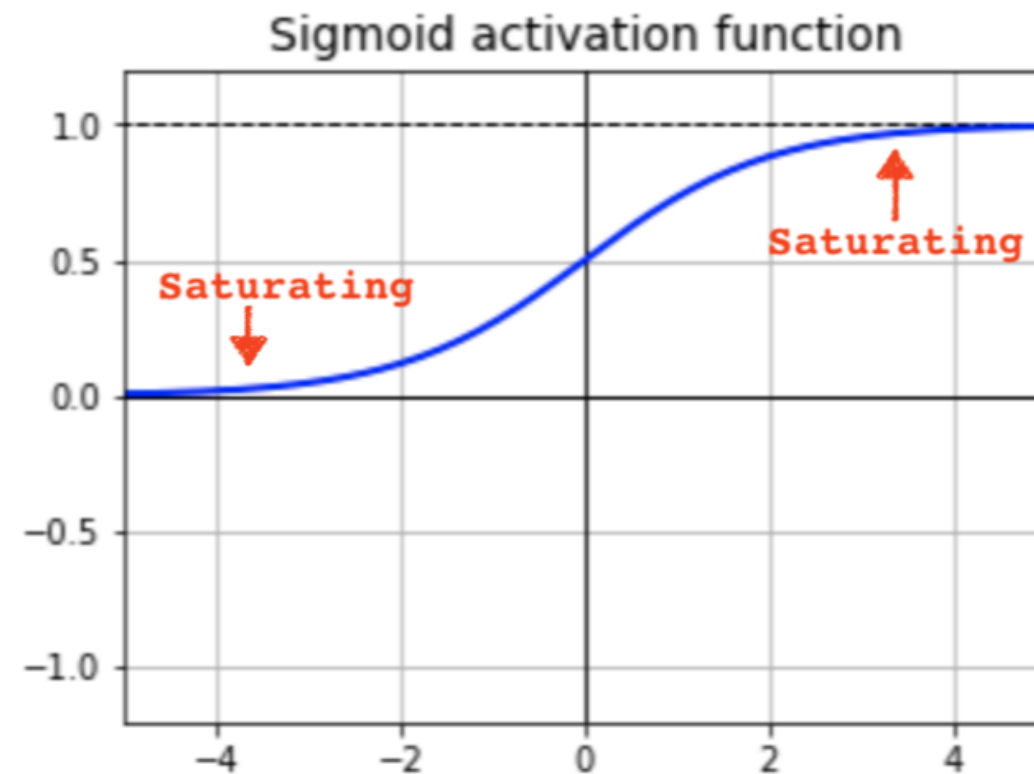
PROs

- Centrada en 0

CONs

- Desvanecimiento de gradiente: a valores grandes, poca diferencia
- Operación computacionalmente mucho más costosa

Función Tangente Hiperbólica



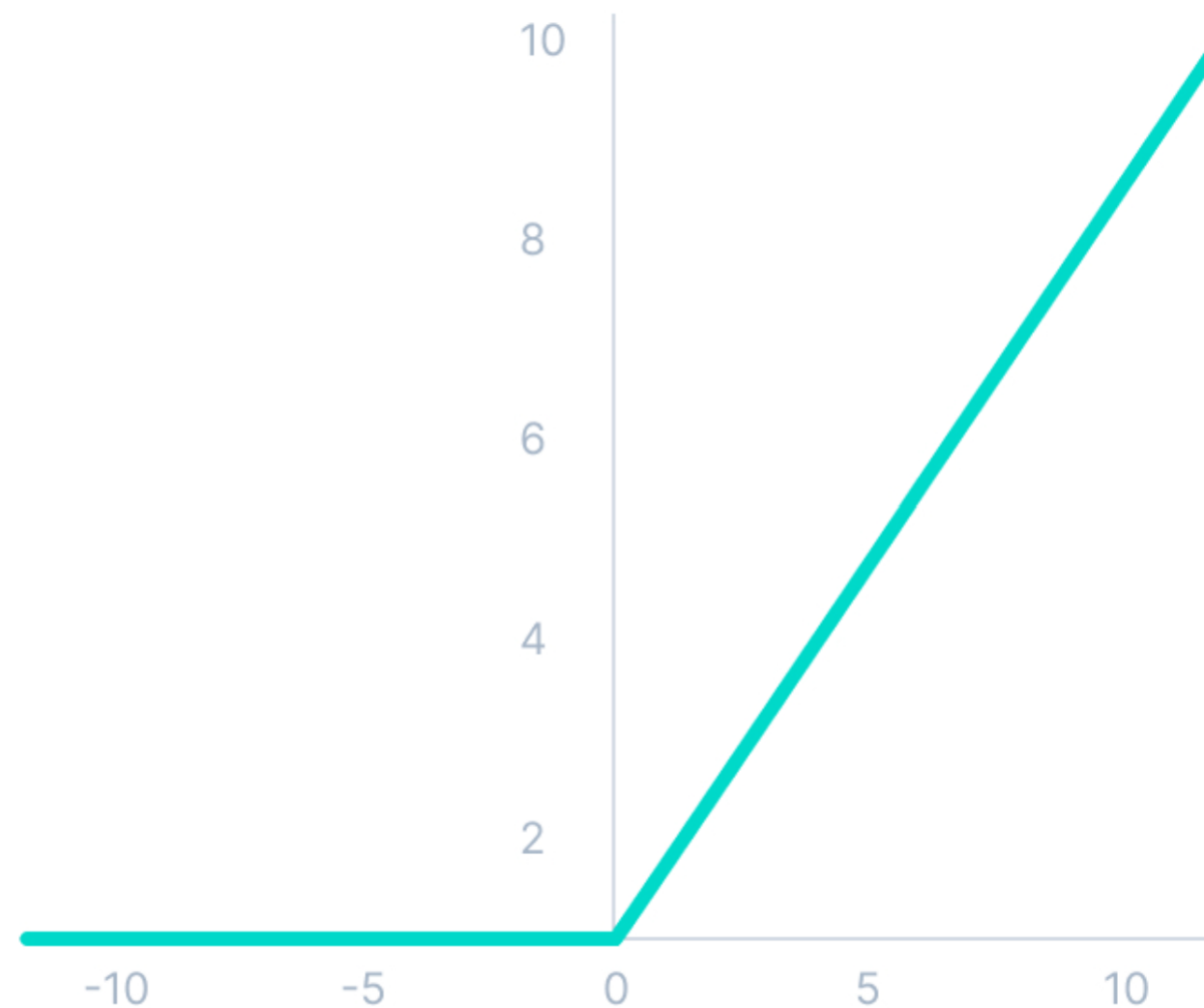
Función Tangente Hiperbólica

Desvanecimiento del gradiente:

- El gradiente de la función de pérdida se propaga hacia atrás multiplicando muchas derivadas.
- Si la derivada es pequeña, implica que se reciben gradientes muy pequeños.
- Por lo tanto, los pesos de esas capas CASI NO SE ACTUALIZAN.
- Esto hace que la red aprenda muy lentamente.

ReLU

$$f(x) = \max(0, x)$$



PROs

- No satura en valores positivos
- Muy eficiente computacionalmente
- Con SGD, converge muy rápido

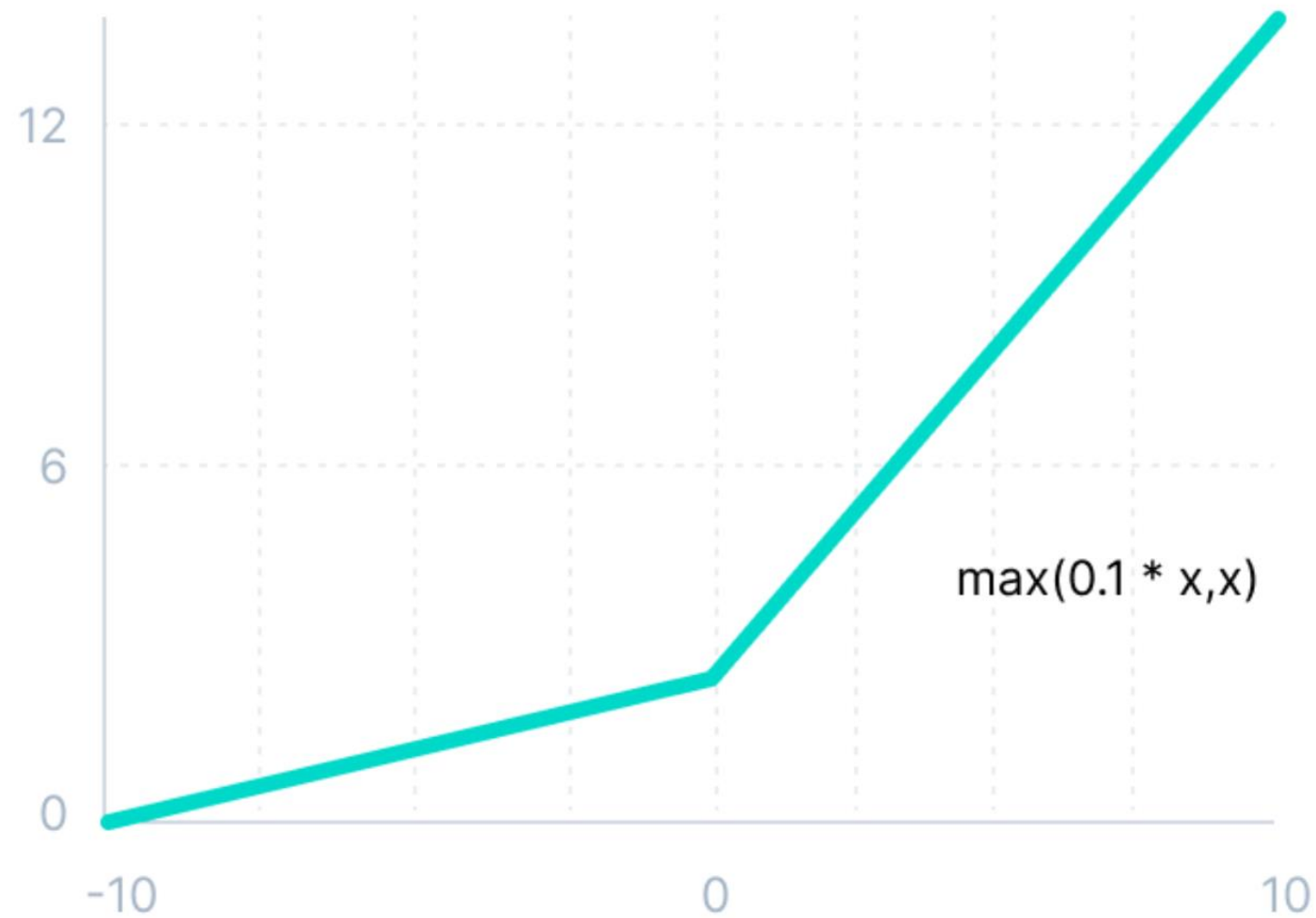
CONs

- No centrada en 0
- Muerte neuronal

Otras ReLU

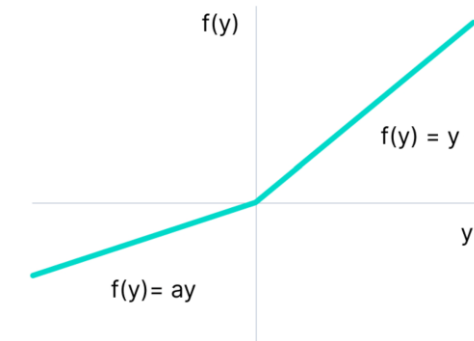
Leaky ReLU

$$\max(0.1 * x, x)$$



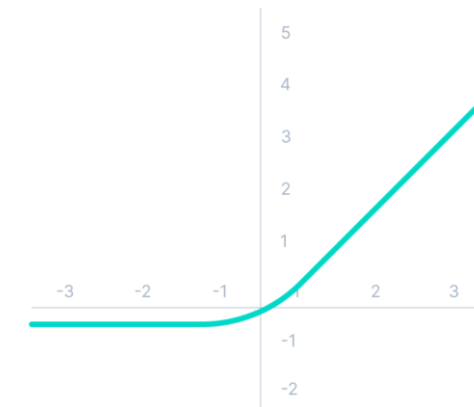
Parametric ReLU

$$f(x) = \max(ax, x)$$



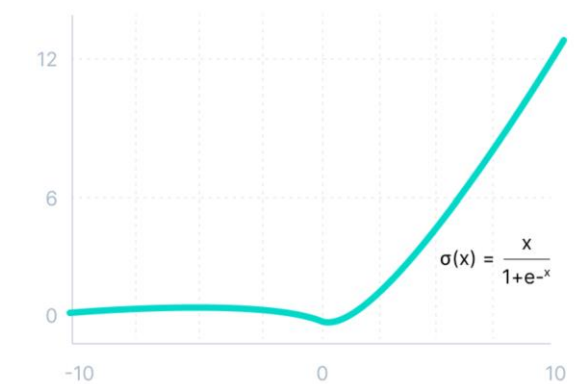
ELU

$$\begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$$

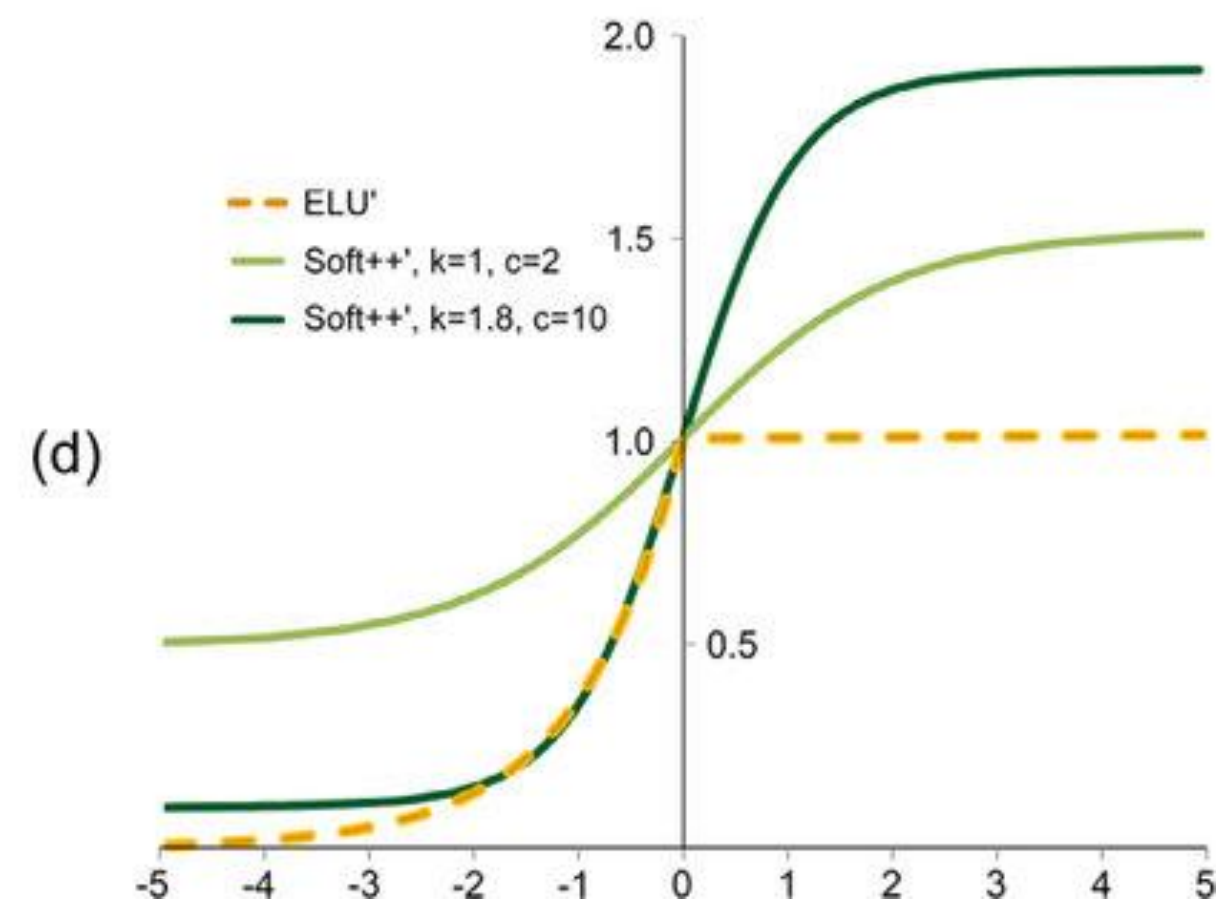
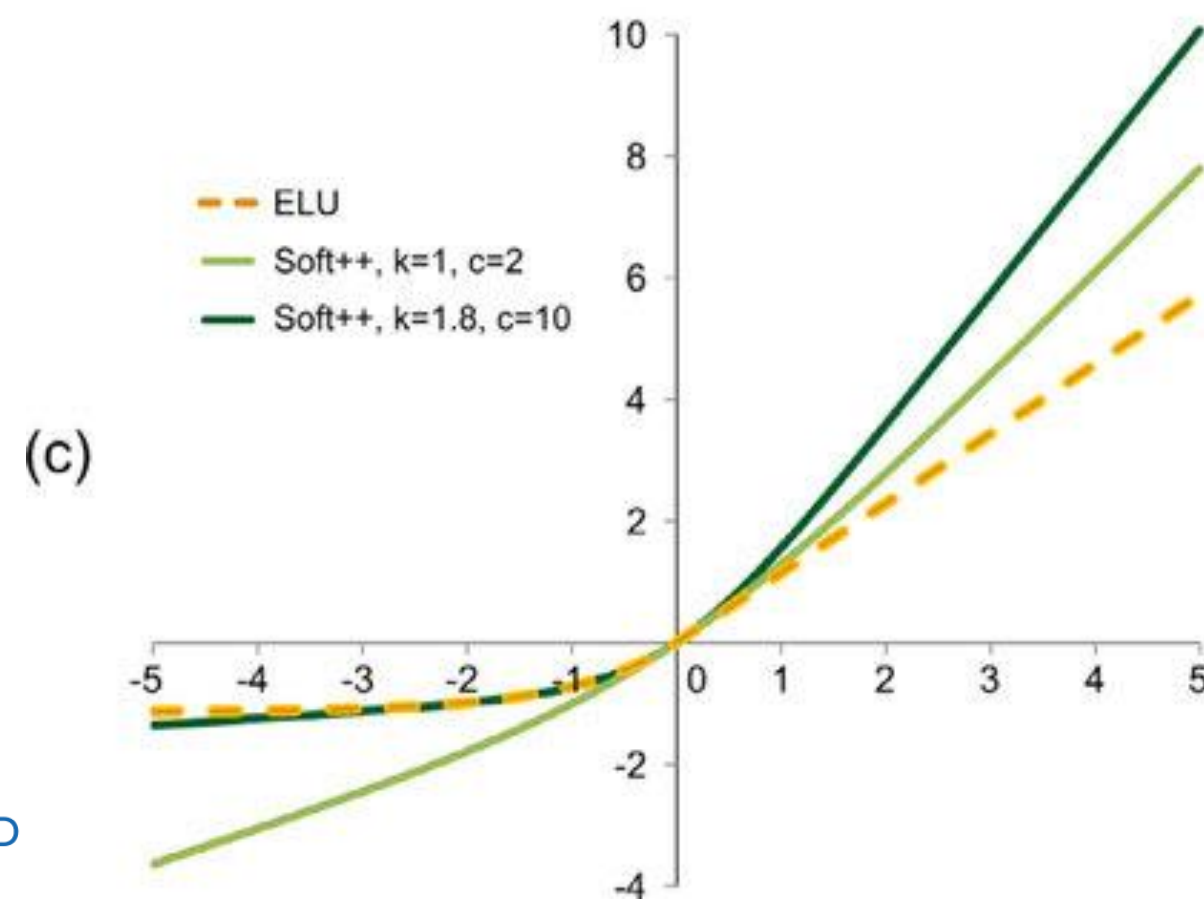
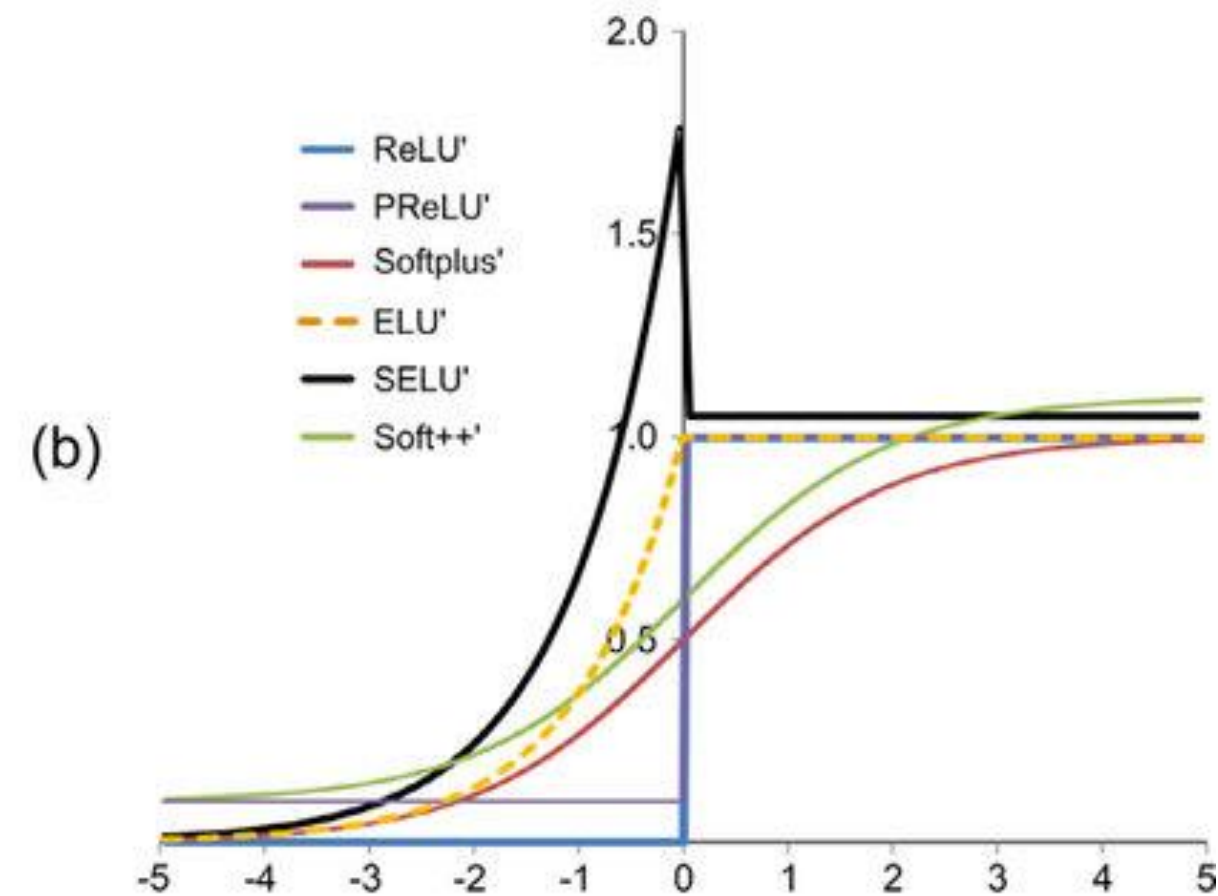
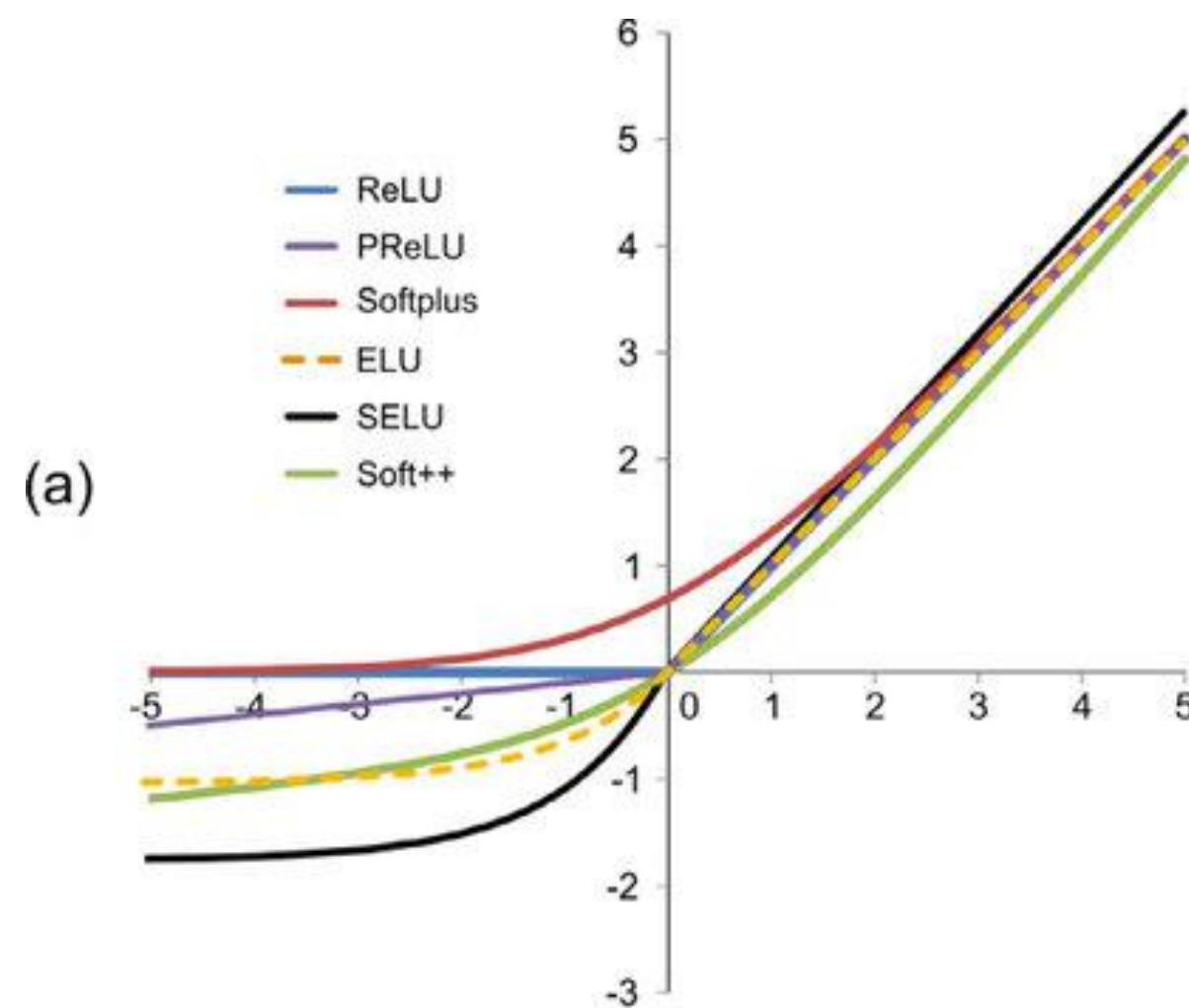


Swift

$$f(x) = x * \text{sigmoid}(x)$$



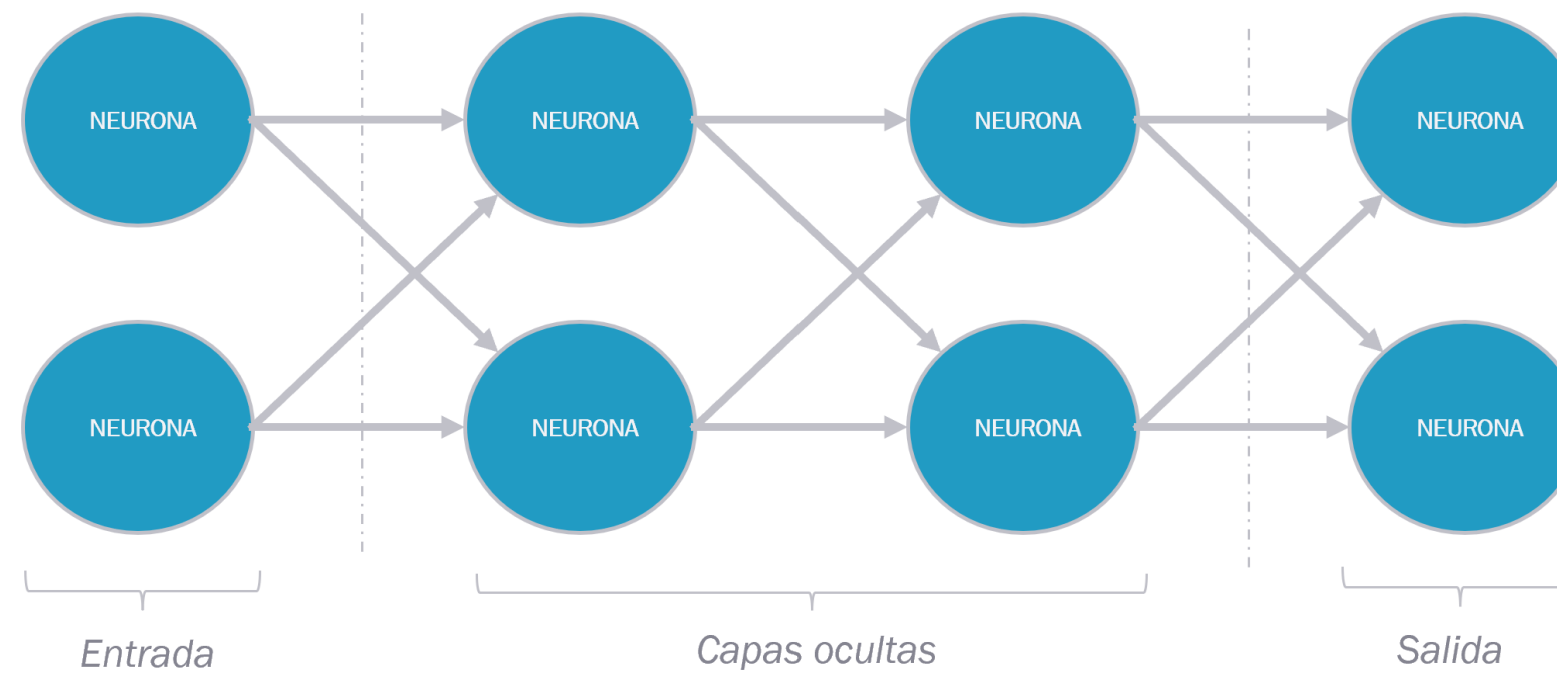
Otras ReLU



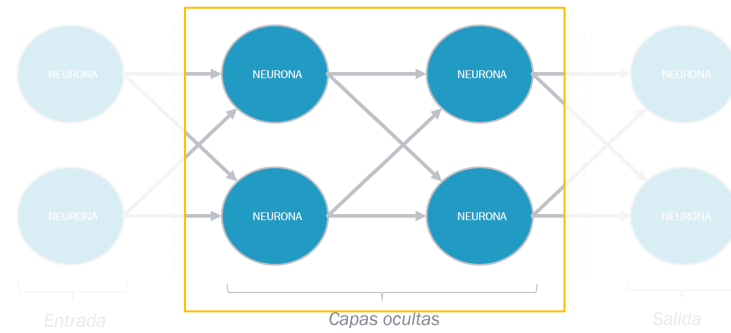
Entonces, ¿cuál utilizo?

Pues...

Depende

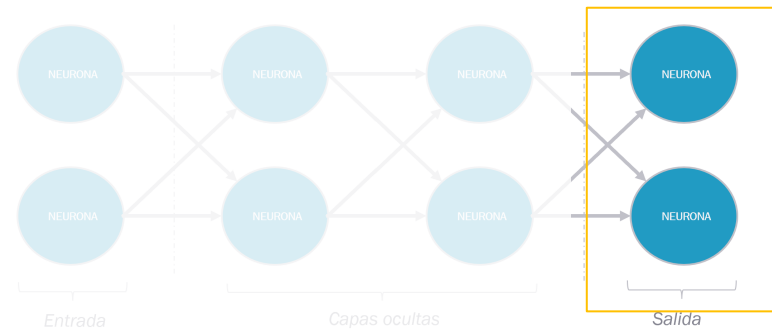


Capas ocultas



- Las funciones sigmoides y derivadas tienen el problema del desvanecimiento de gradiente
- ReLU es la más extendida en las capas ocultas
- En caso de muerte neuronal, se utiliza Leaky ReLU o similar.

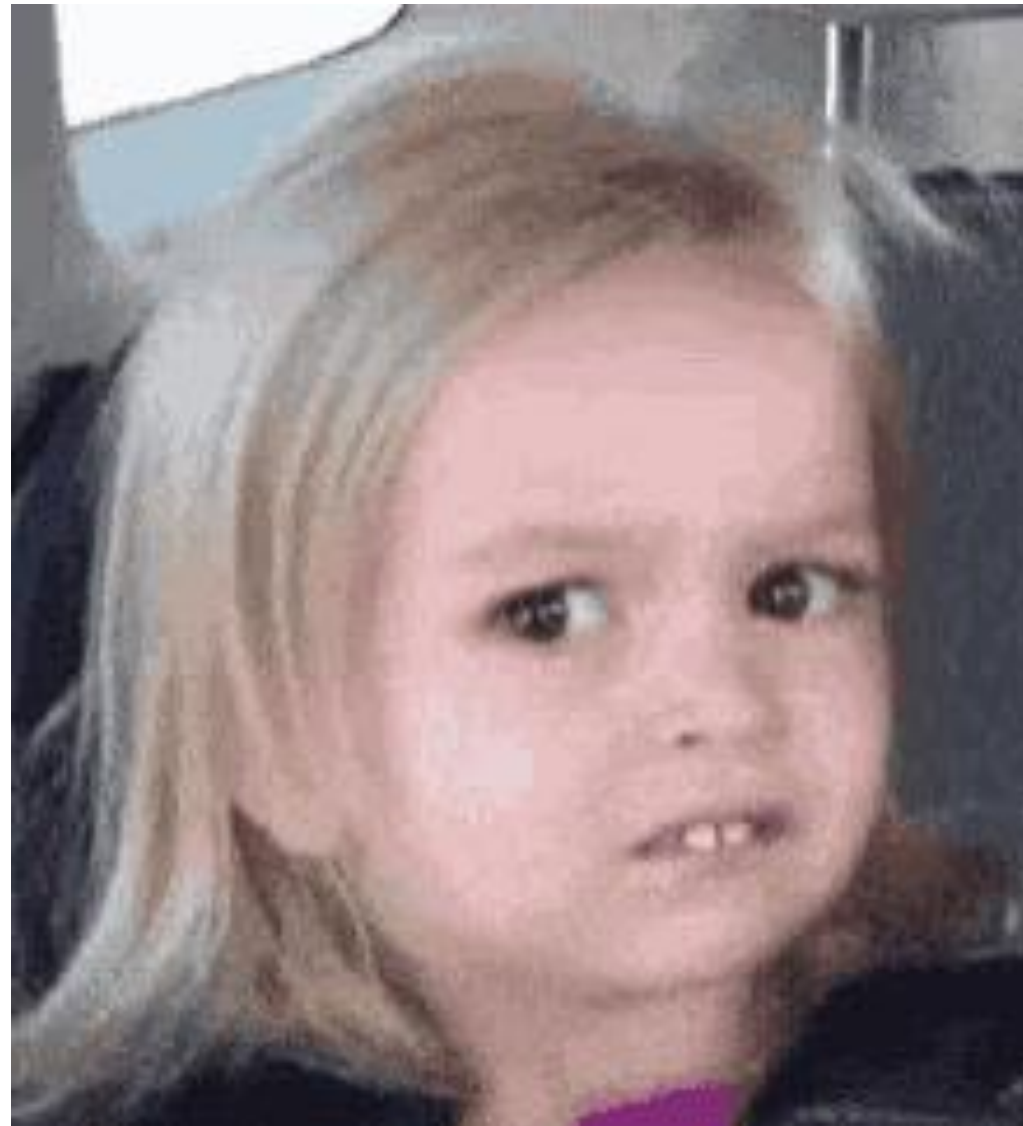
Capa de salida



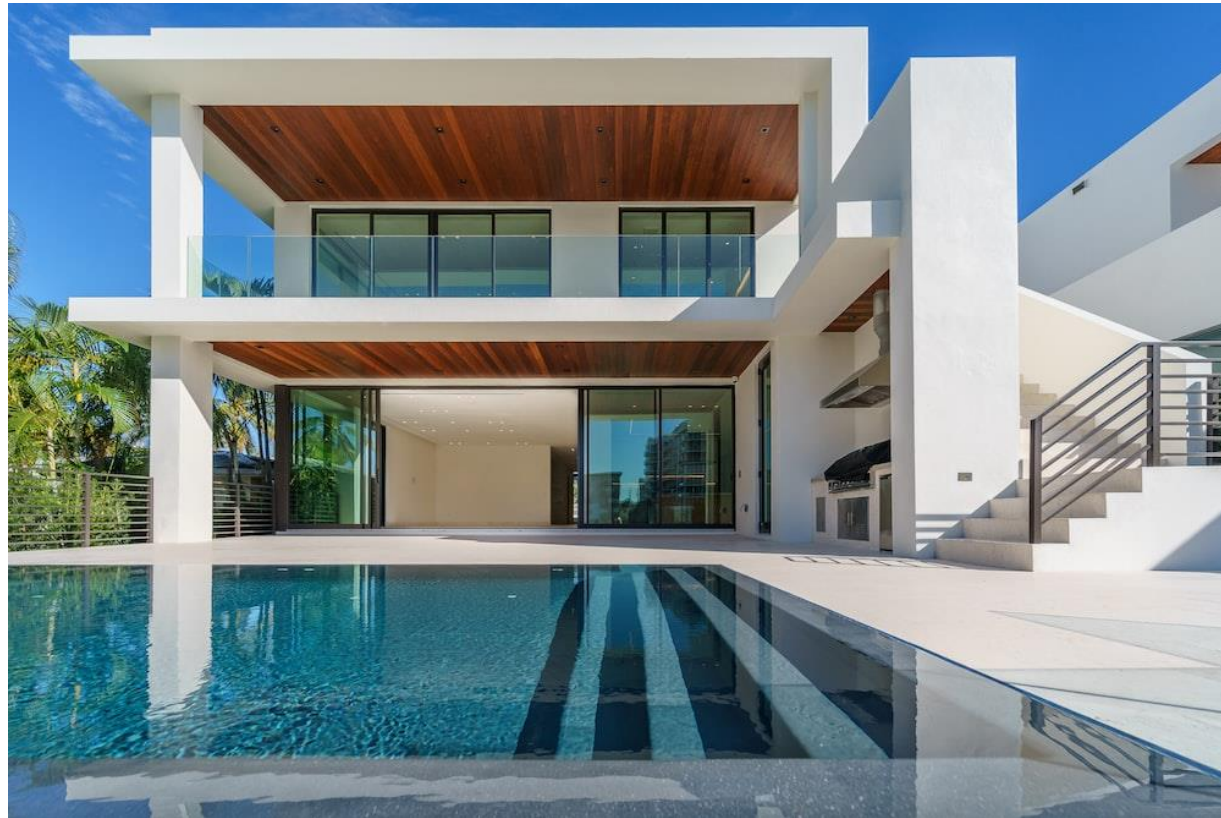
Entonces, ¿cuál utilizo?

Pues...

Depende



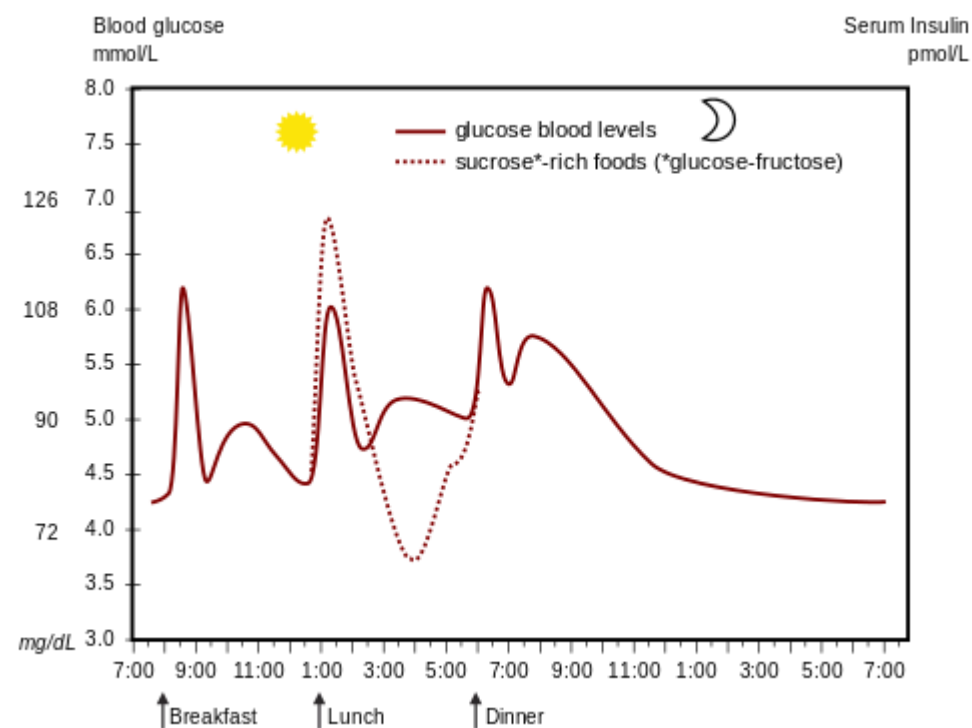
Regresión



Caso: predecir un valor numérico

Función de activación: lineal, ReLU

Nº de neuronas: 1



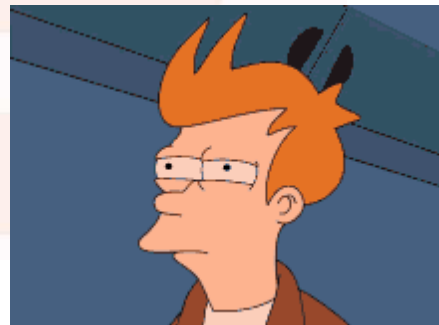
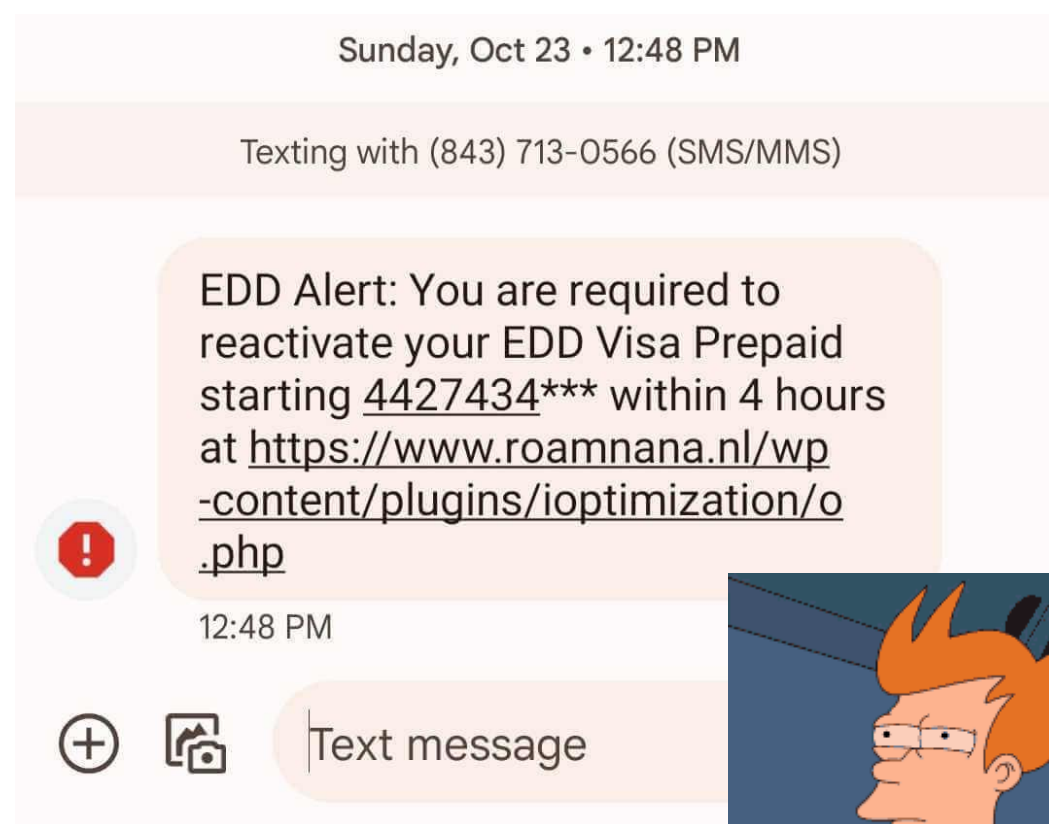
Clasificación binaria



Caso: elegir entre dos estados

Función de activación: **Sigmoide**

Nº de neuronas: **1**



Clasificación Multiclase



Caso: asignar una categoría

Función de activación: **Softmax**

Nº de neuronas: **tantas como clases**



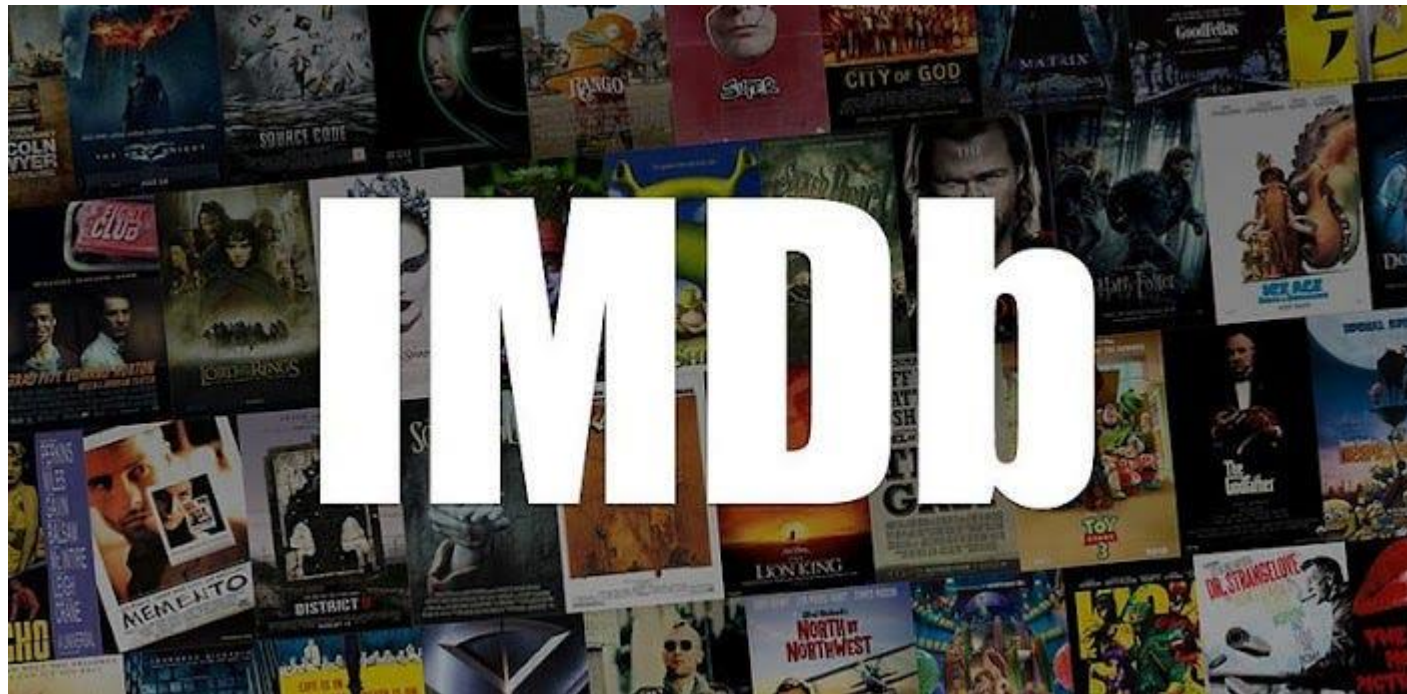
Clasificación Multilabel



Caso: identificar todas las categorías posibles

Función de activación: Sigmoid

Nº de neuronas: tantas como clases



02

Inicialización de parámetros

Inicialización a 0

- Todos los parámetros tienen el mismo valor, por tanto tienen el mismo efecto en la entrada.
- El gradiente de los pesos es siempre el mismo, y por tanto se comporta siempre de la misma manera.
- Conclusión: la red no va a aprender

DEMO:
deeplearning.ai

Inicialización aleatoria

- Se establecen pesos aleatorios, centrados en 0 con una desviación pequeña
- Funciona bien con distintas distribuciones (uniforme, gaussiana)
- A mayor profundidad de capas, puede provocar muerte de gradiente.

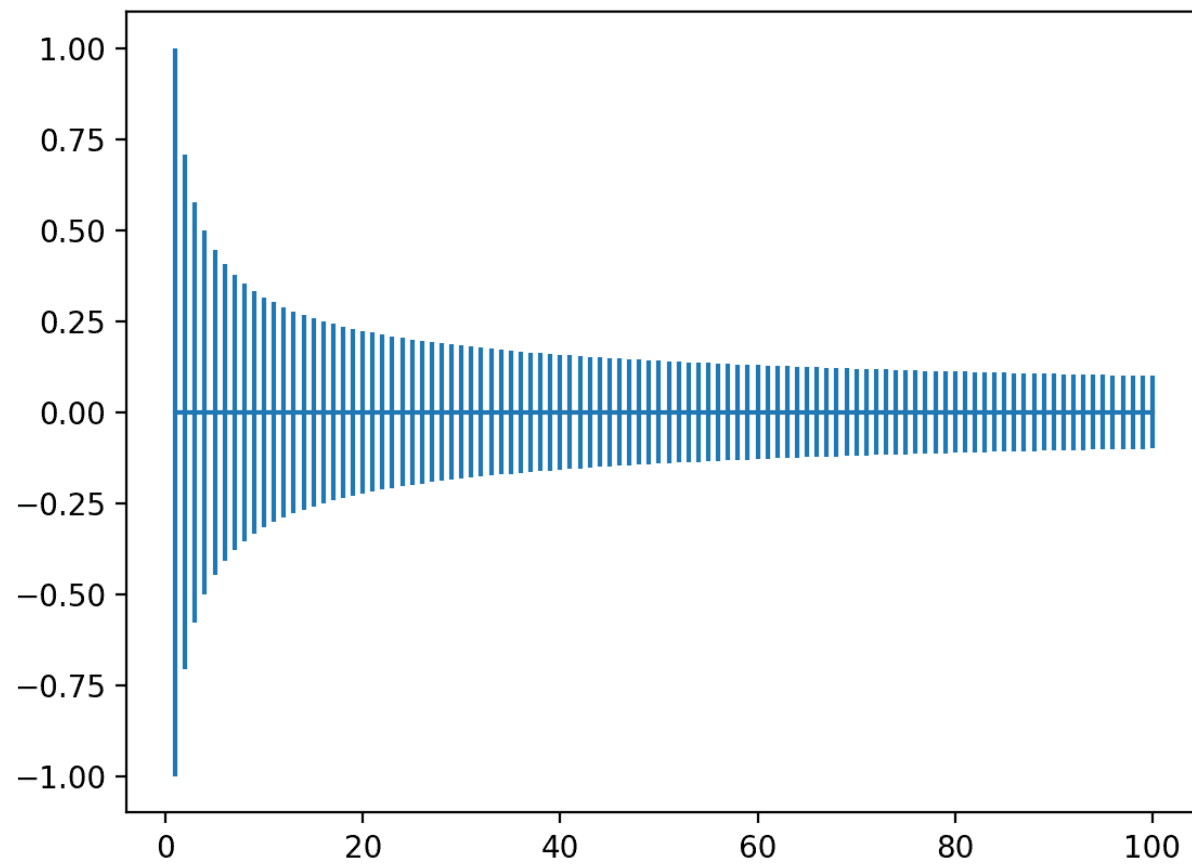
Inicialización Xabier

- Hasta ahora la distribución de los pesos es aleatorio, independiente de los datos.
- Se plantea una relación entre las entradas y las salidas para las neuronas, de manera que su varianza se la misma.

Inicialización Xabier

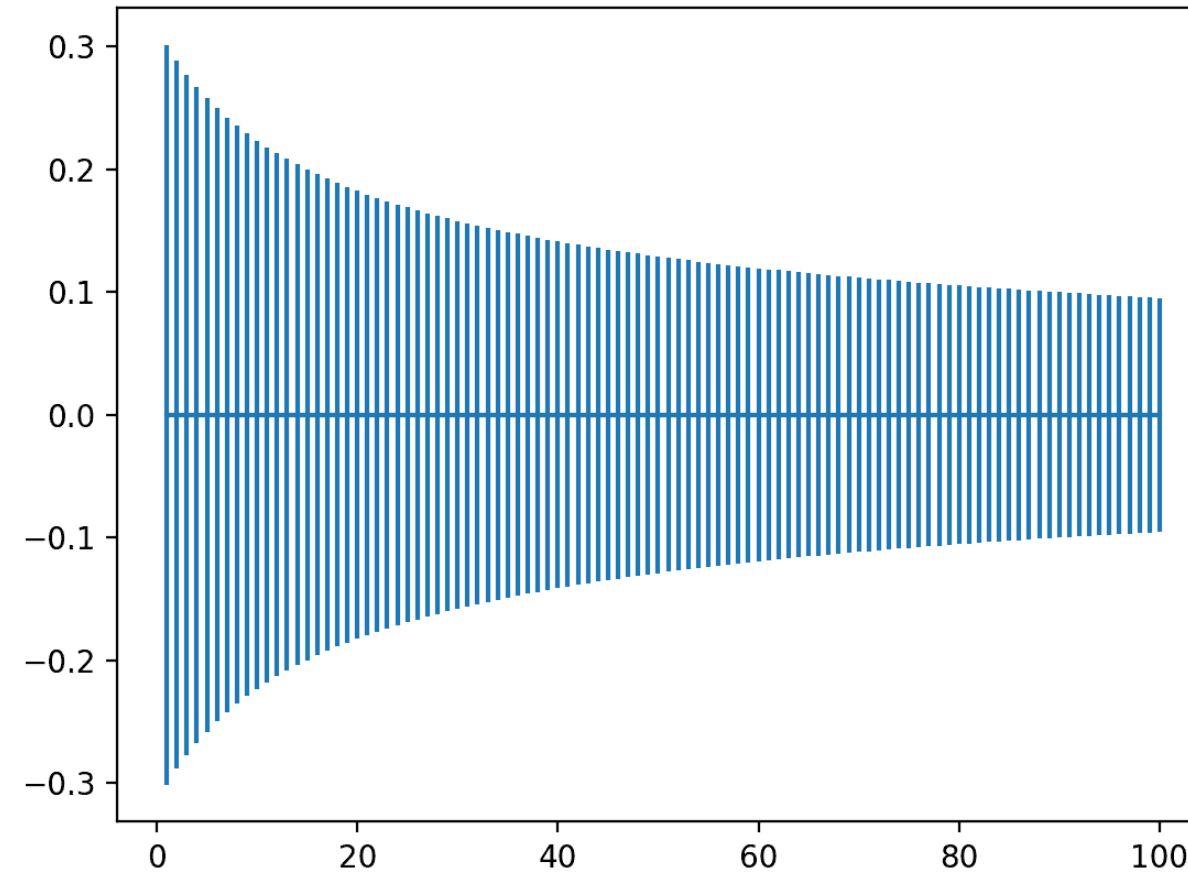
Xavier

$$[-x, x] \text{ for } x = \sqrt{\frac{6}{inputs + outputs}}$$



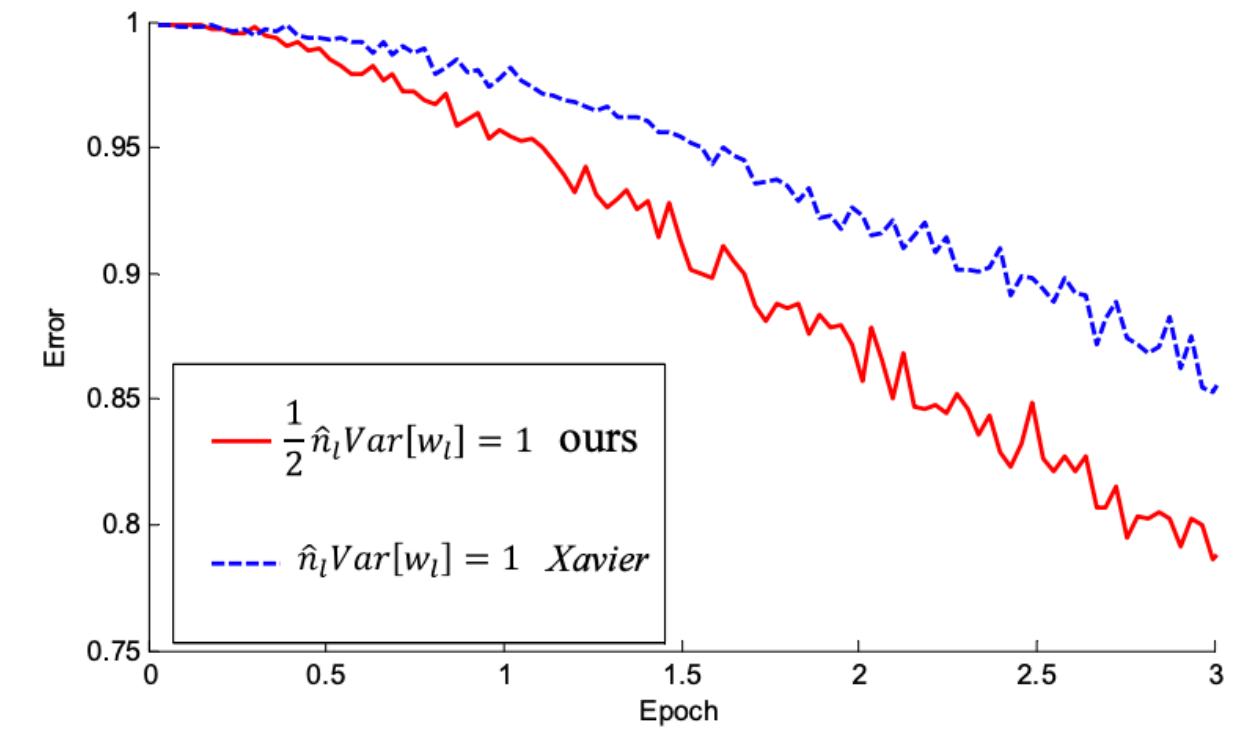
Normalized Xavier

$$\sigma = \sqrt{\frac{2}{inputs + outputs}}$$



He initialization

$$\sigma = \sqrt{2/n_l}$$



Inicialización de sesgos



- La inicialización de los sesgos b no es un elemento crítico a la hora de entrenar una red neuronal.
- Una vez los pesos w son inicializados aleatoriamente, la simetría de la red se rompe.
- Es común inicializarlos a 0 (default en Keras)

03

Batch Normalization

Batch Normalization

Ciente 1	Ciente 2
Edad: 34 años	Edad: 30 años

Distancia Euclídea

$$\text{Dis}(x_1, x_2) = \sqrt{\sum_{i=0}^n (x_{1i} - x_{2i})^2}$$

$$\text{Dis}(x_1, x_2) = \sqrt{(34 - 30)^2} = 4$$

Batch Normalization

Cliente 1
Edad: 34 años
Ingresos: 45.000€

Cliente 2
Edad: 30 años
Ingresos: 16.800€

Distancia Euclídea

$$\sqrt{\sum_{i=0}^n (x_{1i} - x_{2i})^2}$$

$$\sqrt{(34 - 30)^2 + (45000 - 16800)^2} = \sqrt{16 + 795.240.000} \approx 28200$$

Batch Normalization

- Batch normalization consiste en forzar que la entrada de las funciones de activación en todas las capas siga una distribución normal estándar
- Mejora el flujo de gradientes (aumenta velocidad convergencia)
- Evita el vanishing gradient
- Permite learning rate mayores
- La dependencia del entrenamiento respecto a la inicialización disminuye
- El uso de batch normalization es muy frecuente

「muchas gracias.」